UNIT I

PRINCIPLES OF OBJECT-ORIENTED PROGRAMMING & CONTROL STRUCTURES

Debugging Solutions, Corrected C++ Programs,
Pseudo-Code, Dry Runs and Expected Outputs

Prepared as a study and assessment reference

SECTION A — DEBUGGING QUESTIONS

Question 1 — Operator Precedence and Unsigned Countdown

1. Error analysis

- Line 7 — Precedence/clarity defect (not necessarily a wrong result for this exact expression): equality (==) has higher precedence than bitwise AND (&). Therefore `num & 1 == 1 && num > 10` is parsed as `num & (1 == 1) && num > 10`, which becomes `(num & 1) && (num > 10)`. Since `(1 == 1)` is true (integer value 1), this still tests the low bit for this particular expression. Parentheses should explicitly show the intended test: `((num & 1) == 1) && (num > 10)`.
- Line 14 — Loop-boundary defect: `unsigned char` cannot represent a negative value. The condition `i >= 0` is always true after the integer promotions used in the comparison. When `i` is zero and decremented, it wraps to its maximum representable value (typically 255 for an 8-bit unsigned char), so the loop does not terminate naturally. The `break` hides this defect rather than fixing the condition.

2. Corrected complete C++ program

```
#include <iostream>

int main() {
    unsigned char num = 15;

    if (((num & 1) == 1) && (num > 10)) {
        std::cout << "Valid Odd Number: "
                  << static_cast<int>(num) << std::endl;
    } else {
        std::cout << "Condition Failed for: "
                  << static_cast<int>(num) << std::endl;
    }

    // A signed counter can safely reach -1 and stop.
    for (int i = 3; i >= 0; --i) {
        std::cout << i << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

3. Dry run and exact output

Step	num / i	Evaluation or action
1	num = 15	15 & 1 = 1; 1 == 1 is true; 15 > 10 is true.
2	if condition	Both conditions are true; print Valid Odd Number: 15.
3	i = 3, 2, 1, 0	Print each value. After i becomes -1, i >= 0 is false.

Expected console output:

```
Valid Odd Number: 15
3 2 1 0
```

Question 2 — Scope Resolution and Variable Shadowing

1. Error analysis

- Line 11 — Compilation error: `:taxRate` is invalid syntax. The global scope resolution operator consists of two colons, `::`. Use `::taxRate` to refer to the global variable.
- Line 16 — Semantic error: `::taxRate` refers to the global variable (0.18), not the outer local variable (0.05). The unary/global scope resolution operator searches the global namespace; it cannot name an enclosing block or function-local variable.
- An inner declaration named `taxRate` hides the outer local variable within that inner block. To use the outer local rate, give the inner corporate rate a different name (recommended), or avoid shadowing. C++ has no equivalent of `::` for selecting an enclosing local scope.

2. Corrected complete C++ program

```
#include <iostream>

double taxRate = 0.18; // Global standard rate: 18%

int main() {
    double itemPrice = 1000.0;
    double localTaxRate = 0.05; // Local subsidized rate: 5%

    std::cout << "Local Rate: " << localTaxRate << std::endl;

    double totalWithGlobalTax =
        itemPrice + (itemPrice * ::taxRate);

    {
        int corporateTaxPercent = 12;
        double totalWithLocalTax =
            itemPrice + (itemPrice * localTaxRate);

        std::cout << "Corporate Tax Rate: "
                  << corporateTaxPercent << "%" << std::endl;
        std::cout << "Inner total: "
                  << totalWithLocalTax << std::endl;
    }

    std::cout << "Final Global Calculation: "
              << totalWithGlobalTax << std::endl;

    return 0;
}
```

3. Expected output

```
Local Rate: 0.05
Corporate Tax Rate: 12%
Inner total: 1050
Final Global Calculation: 1180
```

Calculation: local total = 1000 + (1000 × 0.05) = 1050; global total = 1000 + (1000 × 0.18) = 1180.

Question 3 — Switch-Case Initialization and Case Labels

1. Error analysis

- Case 1 / case 2 declarations — Compilation error: switch labels do not create separate scopes. Without braces, control can jump to a later case label and bypass initialization of a variable declared earlier in the switch body. This is the cross-initialization problem. Put each case's declarations inside its own `{ }` block.
- `case 4.5` — Compilation error: a switch expression must have an integral or enumeration type after integral promotions. A case label must be an integral constant expression that can be converted to the promoted switch type. A floating-point literal such as 4.5 is not a legal case label.
- Case 2 — Logic defect: the missing `break` causes fall-through into case 3, adding another 100 to the token value. Add `break` after the case 2 calculation.

2. Corrected complete C++ program

```
#include <iostream>

int main() {
    int tokenType = 2;
    float tokenValue = 0.0f;

    switch (tokenType) {
        case 1: {
            int offset = 10;
            tokenValue = 50.0f + offset;
            break;
        }

        case 2: {
            int multiplier = 5;
            tokenValue = 20.0f * multiplier;
            break;
        }

        case 3: {
            tokenValue += 100.0f;
            break;
        }

        // Use an integer code for this token category.
        // Floating-point case labels are not permitted.
        case 4: {
            std::cout << "Floating-point token category detected"
                      << std::endl;
            break;
        }

        default:
            std::cout << "Unknown token" << std::endl;
    }

    std::cout << "Calculated Value: " << tokenValue << std::endl;
    return 0;
}
```

3. Dry run and expected output

Step	Action	tokenValue
1	tokenType = 2; enter case 2.	0.0

2	multiplier = 5; calculate 20.0 × 5.	100.0
3	break exits the switch; case 3 is not executed.	100.0

```
Calculated Value: 100
```

The corrected example assigns integer case 4 as the category code for the floating-point-token category. The switch selector remains an integer.

Question 4 — Reference Initialization and Const Binding

1. Error analysis

- Line 7 — Compilation error: a reference must be initialized when it is declared. `int &refA;` creates an unbound reference, which C++ does not allow.
- Line 11 — Compilation error: a non-const lvalue reference (`int&`) cannot bind to an rvalue literal such as 100. A const lvalue reference can bind to a temporary.
- Line 14 — Compilation error: `pi` is a double lvalue. Binding `int&` would require a conversion to a temporary int, and a non-const lvalue reference cannot bind to that temporary. Use `double&` for the original variable, or `const int&` if a converted temporary is intentionally needed.
- Line 18 — Compilation error: `constRef` refers to a const int and cannot be modified through that reference.

Lifetime extension: when a local const lvalue reference is directly bound to a temporary object, the temporary's lifetime is extended to the lifetime of that reference. Thus `const int& r = 50;` is valid and the temporary int remains alive until `r` goes out of scope. The reference does not make the temporary modifiable.

2. Corrected complete C++ program

```
#include <iostream>

int main() {
    int a = 25;
    int& refA = a;                 // Alias to an existing int

    const int& refLiteral = 100;  // Const reference to temporary

    double pi = 3.14159;
    double& refPi = pi;           // Same type; valid lvalue binding

    const int& constRef = 50;    // Temporary lifetime is extended

    // Modify the original variable through its non-const reference.
    refA = 30;
    refPi = 3.14;

    std::cout << "a and refA: " << a << " " << refA << std::endl;
    std::cout << "refLiteral: " << refLiteral << std::endl;
    std::cout << "refPi: " << refPi << std::endl;
    std::cout << "constRef: " << constRef << std::endl;

    return 0;
}
```

3. Expected output

```
a and refA: 30 30
refLiteral: 100
```

```
refPi: 3.14
constRef: 50
```

Question 5 — Dynamic Array Allocation and Stray Semicolon

1. Error analysis

- Line 10 — Logic/scope defect: the semicolon immediately after the `for` header makes the loop body an empty statement. The following brace block is not the loop body. Also, `i` was declared in the for-init statement and is out of scope after the loop, so `readings[i]` causes a compilation error.
- Line 21 — Undefined behavior: memory allocated with `new[]` must be released with `delete[]`. Using scalar `delete` for an array allocation is a mismatched deallocation and results in undefined behavior.
- `delete` releases one object allocated with `new`; `delete[]` releases an array allocated with `new[]`. The allocation and deallocation forms must match.

2. Corrected complete C++ program

```
#include <iostream>

int main() {
    const int size = 5;
    int* readings = new int[size];

    for (int i = 0; i < size; ++i) {
        readings[i] = (i + 1) * 10;
    }

    int total = 0;
    for (int i = 0; i < size; ++i) {
        total += readings[i];
    }

    std::cout << "Readings: ";
    for (int i = 0; i < size; ++i) {
        std::cout << readings[i] << " ";
    }
    std::cout << std::endl;

    std::cout << "Total: " << total << std::endl;

    delete[] readings;
    readings = nullptr;

    return 0;
}
```

3. Dry run and expected output

i	Assigned reading	Running total
0	10	10
1	20	30
2	30	60
3	40	100
4	50	150

```
Readings: 10 20 30 40 50
Total: 150
```

Debugging Solutions and Pseudo-Code Answers

SECTION B — PSEUDO-CODE WRITING QUESTIONS

The following pseudo-code uses the requested uppercase control keywords and explicit data types. Monetary examples use the currency units shown in the question; the algorithmic structure is independent of a particular country's tax law.

Question 1 — WaterConsumer: Procedural-to-OOP Transformation

1. Design overview

- WaterConsumer encapsulates all consumer and billing data as PRIVATE attributes.
- Initialize validates readings before accepting them. Consumption is current reading minus previous reading.
- ComputeBill applies progressive usage tiers: first 100, next 200, and remaining units. A 10% surcharge applies only when consumption exceeds 300 units, as stated in the requirement.
- DisplayBill prints the invoice. Main creates an object, initializes it, and calls the public methods.

2. Structured pseudo-code

```
CLASS WaterConsumer
    PRIVATE:
        consumerID : STRING
        previousReading : REAL
        currentReading : REAL
        unitsConsumed : REAL
        totalBill : REAL
        isValid : BOOLEAN

    PUBLIC:
        CONSTRUCTOR Initialize(id : STRING, prev : REAL, curr : REAL)
            IF prev < 0 OR curr < 0 OR curr < prev THEN
                isValid <- FALSE
                DISPLAY "ERROR: Invalid meter readings"
            ELSE
                consumerID <- id
                previousReading <- prev
                currentReading <- curr
                unitsConsumed <- 0.0
                totalBill <- 0.0
                isValid <- TRUE
            ENDIF
        END CONSTRUCTOR

        METHOD CalculateConsumption()
            IF isValid = TRUE THEN
                unitsConsumed <- currentReading - previousReading
            ENDIF
        END METHOD

        METHOD ComputeBill()
            DECLARE firstTier : REAL
            DECLARE secondTier : REAL
            DECLARE thirdTier : REAL
            DECLARE surcharge : REAL
            DECLARE baseBill : REAL

            IF isValid = FALSE THEN
                RETURN
            ENDIF

            firstTier <- MIN(unitsConsumed, 100)
            secondTier <- MIN(MAX(unitsConsumed - 100, 0), 200)
```

```
            thirdTier <- MAX(unitsConsumed - 300, 0)

            baseBill <- (firstTier * 1.50)
                        + (secondTier * 2.50)
                        + (thirdTier * 4.00)

            surcharge <- 0.0
            IF unitsConsumed > 300 THEN
                surcharge <- baseBill * 0.10
            ENDIF

            totalBill <- baseBill + surcharge
        END METHOD

        METHOD DisplayBill()
            IF isValid = FALSE THEN
                DISPLAY "Invoice unavailable"
                RETURN
            ENDIF

            DISPLAY "WATER CONSUMPTION INVOICE"
            DISPLAY "Consumer ID: ", consumerID
            DISPLAY "Previous Reading: ", previousReading
            DISPLAY "Current Reading: ", currentReading
            DISPLAY "Units Consumed: ", unitsConsumed
            DISPLAY "Base Bill: ", baseBill
            DISPLAY "Environmental Surcharge: ", surcharge
            DISPLAY "Total Payable: ", totalBill
        END METHOD
ENDCLASS

BEGIN
    DECLARE consumer : WaterConsumer
    consumer.Initialize("WC1001", 1450.0, 1820.0)
    consumer.CalculateConsumption()
    consumer.ComputeBill()
    consumer.DisplayBill()
END
```

Implementation note: `baseBill` and `surcharge` are shown as local calculation values in ComputeBill for readability. In a fully implemented class, store them as private attributes if DisplayBill must access them directly.

3. Dry-run trace

Stage	Calculation	Result
Input	Previous = 1450.0; Current = 1820.0	Valid readings
Consumption	1820.0 − 1450.0	370 units
Tier 1	100 × 1.50	150.00
Tier 2	200 × 2.50	500.00
Tier 3	70 × 4.00	280.00
Base bill	150 + 500 + 280	930.00
Surcharge	10% × 930 (because 370 > 300)	93.00
Final bill	930 + 93	1023.00

Expected invoice total: 1023.00.

Question 2 — Multi-Tier Tax and Precision Cast Engine

1. Design overview

- Cap each eligible deduction independently before subtracting it from gross salary.
- Clamp taxable income to zero so deductions cannot create negative taxable income.
- Calculate tax progressively: each rate applies only to the income slice within its bracket.
- Add 4% cess to raw tax, then round to the nearest whole rupee using INTEGER conversion of tax + 0.5.

2. Structured pseudo-code

```
CLASS TaxCalculationEngine
    PRIVATE:
        grossAnnualSalary : REAL
        deduction80C : REAL
        deductionHealthInsurance : REAL
        eligible80C : REAL
        eligibleHealth : REAL
        taxableIncome : REAL
        rawTax : REAL
        cess : REAL
        finalTax : REAL

    PUBLIC:
        METHOD Calculate(gross : REAL, d80C : REAL, dHealth : REAL)
            grossAnnualSalary <- MAX(gross, 0)
            eligible80C <- MIN(MAX(d80C, 0), 150000)
            eligibleHealth <- MIN(MAX(dHealth, 0), 25000)

            taxableIncome <- grossAnnualSalary
                             - eligible80C
                             - eligibleHealth
            taxableIncome <- MAX(taxableIncome, 0)

            rawTax <- 0.0

            IF taxableIncome <= 250000 THEN
                rawTax <- 0.0
            ELSE IF taxableIncome <= 500000 THEN
                rawTax <- (taxableIncome - 250000) * 0.05
            ELSE IF taxableIncome <= 1000000 THEN
                rawTax <- (250000 * 0.05)
                          + ((taxableIncome - 500000) * 0.20)
            ELSE
                rawTax <- (250000 * 0.05)
                          + (500000 * 0.20)
                          + ((taxableIncome - 1000000) * 0.30)
            ENDIF

            cess <- rawTax * 0.04
            finalTax <- INTEGER(rawTax + cess + 0.5)
        END METHOD

        METHOD DisplayResult()
            DISPLAY "Eligible 80C: ", eligible80C
            DISPLAY "Eligible Health Deduction: ", eligibleHealth
            DISPLAY "Taxable Income: ", taxableIncome
            DISPLAY "Raw Tax: ", rawTax
            DISPLAY "Health and Education Cess: ", cess
            DISPLAY "Final Tax (rounded): ", finalTax
        END METHOD
```

```
ENDCLASS

BEGIN
    DECLARE engine : TaxCalculationEngine
    engine.Calculate(1200000, 180000, 30000)
    engine.DisplayResult()
END
```

3. Dry-run trace

Stage	Calculation	Result
Gross salary	Given	1,200,000
Eligible 80C	MIN(180,000, 150,000)	150,000
Eligible health	MIN(30,000, 25,000)	25,000
Taxable income	1,200,000 − 150,000 − 25,000	1,025,000
0% slab	First 250,000 × 0%	0
5% slab	250,000 × 5%	12,500
20% slab	500,000 × 20%	100,000
30% slab	25,000 × 30%	7,500
Raw tax	12,500 + 100,000 + 7,500	120,000
Cess	120,000 × 4%	4,800
Final tax	INTEGER(124,800 + 0.5)	124,800

Final rounded tax = 124,800.

Question 3 — Bitwise Mask Validation and UAV Flight Readiness

1. Required masks

Constant	Hex value	Meaning
MANDATORY_MASK	0x0F	Bits 0–3: all four essential sensors must be ON
INHIBITOR_MASK	0xB0	Bits 4, 5 and 7: all must be OFF
GYRO_MASK	0x01	Gyroscope locked
ACCEL_MASK	0x02	Accelerometer calibrated
BARO_MASK	0x04	Altimeter online
GPS_MASK	0x08	GPS fix acquired
LOW_BATTERY_MASK	0x10	Low battery warning
OVER_TEMP_MASK	0x20	Over-temperature warning
RADIO_MASK	0x40	Radio link active; non-blocking
ACTUATOR_FAIL_MASK	0x80	Fatal actuator failure

2. Structured pseudo-code

```
CONSTANT MANDATORY_MASK : BYTE <- 0x0F
CONSTANT LOW_BATTERY_MASK : BYTE <- 0x10
CONSTANT OVER_TEMP_MASK : BYTE <- 0x20
CONSTANT ACTUATOR_FAIL_MASK : BYTE <- 0x80
CONSTANT INHIBITOR_MASK : BYTE <- 0xB0

METHOD CheckFlightReadiness(statusRegister : BYTE)
    DECLARE missingMask : BYTE
    DECLARE activeInhibitors : BYTE
    DECLARE ready : BOOLEAN

    ready <- TRUE
    activeInhibitors <- statusRegister AND INHIBITOR_MASK

    IF (activeInhibitors AND LOW_BATTERY_MASK) <> 0 THEN
        DISPLAY "ERROR: Low battery warning (bit 4)"
```

```
        ready <- FALSE
    ENDIF

    IF (activeInhibitors AND OVER_TEMP_MASK) <> 0 THEN
        DISPLAY "ERROR: Over-temperature warning (bit 5)"
        ready <- FALSE
    ENDIF

    IF (activeInhibitors AND ACTUATOR_FAIL_MASK) <> 0 THEN
        DISPLAY "FATAL: Actuator hardware failure (bit 7)"
        ready <- FALSE
    ENDIF

    missingMask <- MANDATORY_MASK XOR
                   (statusRegister AND MANDATORY_MASK)

    IF missingMask <> 0 THEN
        DISPLAY "ERROR: Missing mandatory sensor bit mask = ", HEX(missingMask)
        IF (missingMask AND 0x01) <> 0 THEN DISPLAY "Missing gyroscope"
        IF (missingMask AND 0x02) <> 0 THEN DISPLAY "Missing accelerometer"
        IF (missingMask AND 0x04) <> 0 THEN DISPLAY "Missing barometric altimeter"
        IF (missingMask AND 0x08) <> 0 THEN DISPLAY "Missing GPS fix"
        ready <- FALSE
    ENDIF

    IF ready = TRUE THEN
        DISPLAY "FLIGHT READY"
    ELSE
        DISPLAY "FLIGHT NOT READY"
    ENDIF
END METHOD
```

The XOR expression isolates missing mandatory bits because the required mask has 1s in every mandatory position, while the masked status register has 1s only for sensors that are present. The NOT operator can also be used to form a missing-bit mask (`(~statusRegister) AND MANDATORY_MASK`); the XOR form is used above as explicitly requested.

3. Test traces

Test byte	Mandatory bits	Inhibitors	Missing mask	Result
0x4F	0x4F AND 0x0F = 0x0F (all present)	0x4F AND 0xB0 = 0x00	0x0F XOR 0x0F = 0x00	FLIGHT READY; radio bit 6 is set but non-blocking
0x3B	0x3B AND 0x0F = 0x0B (bit 2 absent)	0x3B AND 0xB0 = 0x30 (bits 4 and 5 active)	0x0F XOR 0x0B = 0x04	NOT READY; low battery, over-temperature, and missing altimeter

For 0x3B, diagnostics are: low battery warning (bit 4), over-temperature warning (bit 5), missing barometric altimeter (bit 2), then FLIGHT NOT READY.

Question 4 — Divisor and Number Classification Engine

1. Design overview

- Reject non-positive input and request another value. The pseudo-code uses a REPEAT loop; a GOTO-style retry label is shown as an alternative.
- For N = 1, the proper-divisor sum is zero, so the number is deficient.
- Start the proper-divisor sum at 1 for N > 1. Test candidate divisors only up to FLOOR(SQRT(N)); when a divisor is found, add both members of the factor pair, but add a square root only once.

- Use CONTINUE for non-divisors. The number is perfect, abundant, or deficient according to whether the proper-divisor sum equals, exceeds, or is below N.

2. Structured pseudo-code

```
CLASS DivisorAndPrimeEngine
    PUBLIC:
        METHOD Classify()
            DECLARE N : INTEGER
            DECLARE d : INTEGER
            DECLARE pairedDivisor : INTEGER
            DECLARE sumOfDivisors : INTEGER
            DECLARE limit : INTEGER

            REPEAT
                DISPLAY "Enter a positive integer: "
                INPUT N
                IF N <= 0 THEN
                    DISPLAY "Invalid input. Enter a value greater than zero."
                ENDIF
            UNTIL N > 0

            IF N = 1 THEN
                sumOfDivisors <- 0
            ELSE
                sumOfDivisors <- 1
                limit <- FLOOR(SQRT(N))

                FOR d <- 2 TO limit STEP 1
                    IF N MOD d <> 0 THEN
                        CONTINUE
                    ENDIF

                    pairedDivisor <- N / d
                    sumOfDivisors <- sumOfDivisors + d

                    IF pairedDivisor <> d THEN
                        sumOfDivisors <- sumOfDivisors + pairedDivisor
                    ENDIF
                ENDFOR
            ENDIF

            DISPLAY "Sum of proper divisors: ", sumOfDivisors

            IF sumOfDivisors = N THEN
                DISPLAY "Perfect Number"
            ELSE IF sumOfDivisors > N THEN
                DISPLAY "Abundant Number"
            ELSE
                DISPLAY "Deficient Number"
            ENDIF
        END METHOD
ENDCLASS

BEGIN
    DECLARE engine : DivisorAndPrimeEngine
    engine.Classify()
END
```

Complexity: the divisor search performs at most FLOOR(√N) − 1 candidate checks, so its time complexity is O(√N). The algorithm uses O(1) extra space.

3. Dry-run trace for N = 28

d	28 MOD d	Paired divisor	Sum after step
Initial	—	—	1
2	0	14	1 + 2 + 14 = 17
3	1	—	17 (CONTINUE)
4	0	7	17 + 4 + 7 = 28
5	1	—	28 (CONTINUE)

Final sum = 28. Since sumOfDivisors = N, 28 is a PERFECT NUMBER.

4. Dry-run trace for N = 12

d	12 MOD d	Paired divisor	Sum after step
Initial	—	—	1
2	0	6	1 + 2 + 6 = 9
3	0	4	9 + 3 + 4 = 16

Final sum = 16. Since 16 > 12, 12 is an ABUNDANT NUMBER.

Question 5 — State-Machine ATM Controller

1. Design overview

- Authentication permits at most three PIN entries. A correct PIN enters the session; three incorrect entries lock the card and terminate the program.
- A sentinel-controlled WHILE loop continues until the user selects Exit or the card is locked.
- SWITCH selects the transaction. Deposits must be positive. Withdrawals must be positive, no greater than the available balance, and divisible by 100.

2. Structured pseudo-code

```
CLASS ATM_SessionController
    PRIVATE:
        correctPIN : INTEGER <- 4821
        balance : REAL <- 1000.00
        attempts : INTEGER <- 0
        authenticated : BOOLEAN <- FALSE
        sessionActive : BOOLEAN <- FALSE

    PUBLIC:
        METHOD Run()
            DECLARE enteredPIN : INTEGER
            DECLARE option : INTEGER
            DECLARE amount : REAL

            WHILE attempts < 3 AND authenticated = FALSE
                DISPLAY "Enter 4-digit PIN: "
                INPUT enteredPIN
                attempts <- attempts + 1

                IF enteredPIN = correctPIN THEN
                    authenticated <- TRUE
                    DISPLAY "Authentication successful"
                ELSE
                    DISPLAY "Incorrect PIN"
```

```
        ENDIF
    ENDWHILE

    IF authenticated = FALSE THEN
        DISPLAY "Card locked. Too many incorrect attempts."
        RETURN
    ENDIF

    sessionActive <- TRUE

    WHILE sessionActive = TRUE
        DISPLAY "1. Balance Inquiry"
        DISPLAY "2. Cash Deposit"
        DISPLAY "3. Cash Withdrawal"
        DISPLAY "4. Exit"
        INPUT option

        SWITCH option
            CASE 1:
                DISPLAY "Available balance: ", balance
                BREAK

            CASE 2:
                DISPLAY "Enter deposit amount: "
                INPUT amount
                IF amount > 0 THEN
                    balance <- balance + amount
                    DISPLAY "Deposit successful"
                    DISPLAY "New balance: ", balance
                ELSE
                    DISPLAY "Deposit rejected: amount must be positive"
                ENDIF
                BREAK

            CASE 3:
                DISPLAY "Enter withdrawal amount: "
                INPUT amount

                IF amount <= 0 THEN
                    DISPLAY "Withdrawal rejected: amount must be positive"
                ELSE IF amount > balance THEN
                    DISPLAY "Withdrawal rejected: insufficient balance"
                ELSE IF amount MOD 100 <> 0 THEN
                    DISPLAY "Withdrawal rejected: amount must be a multiple of 100"
                ELSE
                    balance <- balance - amount
                    DISPLAY "Withdrawal successful"
                    DISPLAY "Remaining balance: ", balance
                ENDIF
                BREAK

            CASE 4:
                sessionActive <- FALSE
                DISPLAY "Thank you. Session ended."
                BREAK

            DEFAULT:
                DISPLAY "Invalid menu option"
        ENDSWITCH
    ENDWHILE
END METHOD
```

```
ENDCLASS

BEGIN
    DECLARE atm : ATM_SessionController
    atm.Run()
END
```

For a real implementation, use an integer number of currency units (or a fixed-precision decimal type) for money rather than binary floating-point. The pseudo-code uses REAL to match the question's requested data types.

3. Dry-run transaction trace

Assumption for this sample trace: opening balance = 1000.00, as initialized in the pseudo-code.

Step	User input / event	System action	Balance
1	PIN = 1111	Incorrect PIN; attempts = 1	1000.00
2	PIN = 4821	Correct PIN; authentication successful	1000.00
3	Menu option = 3; amount = 250	250 is positive and within balance, but 250 MOD 100 = 50. Reject withdrawal.	1000.00
4	Menu option = 3; amount = 300	300 is positive, within balance, and 300 MOD 100 = 0. Approve; subtract 300.	700.00
5	Menu option = 1	Display available balance.	700.00
6	Menu option = 4	End the session.	700.00

Representative console interaction:

```
Enter 4-digit PIN: 1111
Incorrect PIN
Enter 4-digit PIN: 4821
Authentication successful

3. Cash Withdrawal
Enter withdrawal amount: 250
Withdrawal rejected: amount must be a multiple of 100

3. Cash Withdrawal
Enter withdrawal amount: 300
Withdrawal successful
Remaining balance: 700

1. Balance Inquiry
Available balance: 700
4. Exit
Thank you. Session ended.
```

Quick Revision — Important C++ Rules

Concept	Rule to remember
Operator precedence	Use parentheses around bitwise comparisons, e.g. `((x & mask) != 0)`. Equality binds more tightly than bitwise AND.
Unsigned countdown	Do not use `unsigned` with a condition such as `i >= 0`; it is always true. Use a signed counter or a different loop condition.
Scope resolution	`::name` refers to a global namespace name; it does not access an enclosing local variable.
switch-case	Switch selectors and case labels are integral/enumeration based. Use braces around case-local declarations and `break` to prevent fall-through.
References	A reference must be initialized. Non-const lvalue references cannot bind to literals or conversion temporaries.
Const reference	A const lvalue reference can bind to a temporary and extend its lifetime to the reference's lifetime.
Dynamic arrays	Pair `new[]` with `delete[]`; pair scalar `new` with scalar `delete`.
Progressive slabs	Apply each rate only to the income portion inside that slab.
Bit masks	Use AND to test bits, OR to combine masks, XOR to find differing bits, and NOT to invert bits (then mask unwanted positions).
Divisor pairs	For d dividing N, add d and N/d; when d equals N/d, add the square root only once.

End of Unit I answer document.